

1. Propósito General del Proyecto

El propósito de este proyecto es diseñar e implementar una base de datos utilizando el motor **MariaDB** para gestionar el inventario, los clientes y el flujo de ventas de una **tienda de videojuegos** y tecnología.

El sistema busca solucionar problemas comunes de los negocios tradicionales, tales como:

- La pérdida de información histórica de ventas.
- La duplicación de datos de clientes.
- La falta de control en tiempo real sobre el stock de los productos.
- La inconsistencia en los precios facturados cuando un artículo cambia de valor en góndola.

2. Diagramación Entidad-Relación (ER)

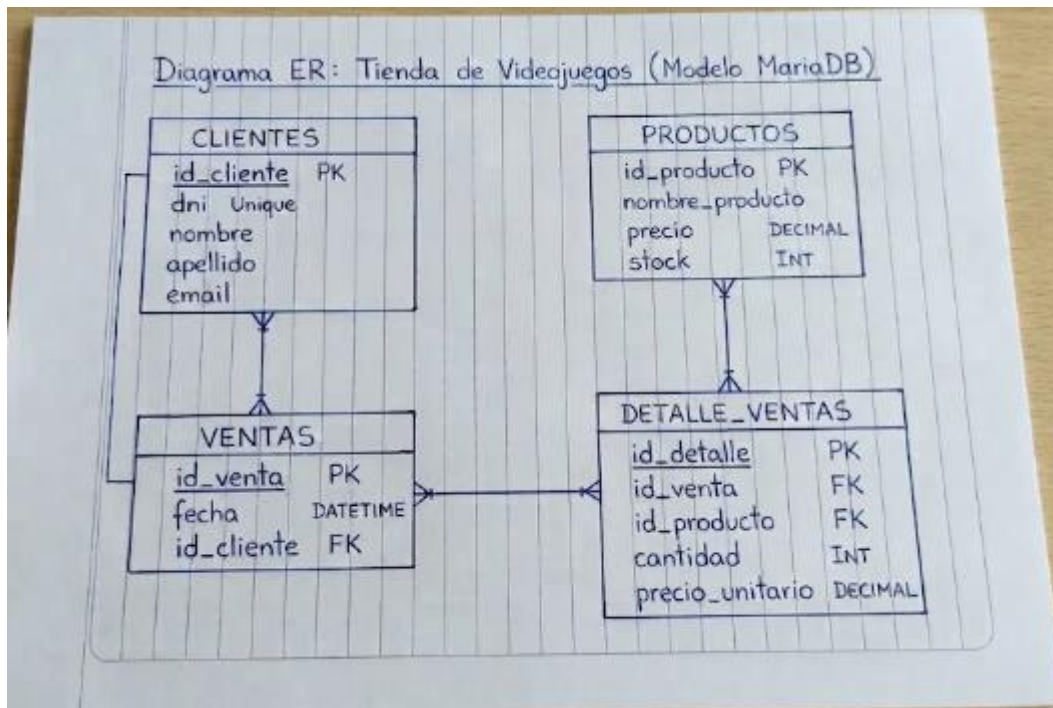


Diagrama Entidad-Relación para el sistema de gestión de la tienda de videojuegos, diseñado para su implementación en MariaDB/heidisql.

Tabla: clientes

Almacena la información de contacto e identificación de los compradores.

- **id_cliente (INT, PK):** Identificador único autoincremental de cada cliente.
- **dni (VARCHAR(20), UNIQUE, NOT NULL):** Documento Nacional de Identidad. No puede repetirse ni quedar vacío.
- **nombre (VARCHAR(100), NOT NULL):** Nombre del cliente.
- **apellido (VARCHAR(100), NOT NULL):** Apellido del cliente.
- **email (VARCHAR(100)):** Correo electrónico (opcional).

Tabla: productos

Contiene el catálogo de artículos disponibles para la venta.

- **id_producto (INT, PK):** Identificador único autoincremental de cada producto.
- **nombre_producto (VARCHAR(100), NOT NULL):** Nombre comercial del artículo.
- **precio (DECIMAL(10,2), NOT NULL):** Precio de venta actual. Se utiliza DECIMAL para garantizar precisión exacta en centavos.
- **stock (INT, NOT NULL):** Cantidad de unidades disponibles en almacén (por defecto inicia en 0).

Tabla: ventas

Funciona como la cabecera de la factura o ticket de compra.

- **id_venta (INT, PK):** Número único de factura autoincremental.
- **fecha (DATETIME, NOT NULL):** Fecha y hora exacta de la transacción (se genera automáticamente con la hora del servidor).
- **id_cliente (INT, FK):** Vincula la venta con el cliente que realizó la compra.

Tabla: detalle_ventas

Es la tabla intermedia que desglosa los artículos comprados en cada ticket.

- **id_detalle (INT, PK):** Identificador único de la línea de la factura.
- **id_venta (INT, FK):** Vincula la línea con su factura correspondiente.
- **id_producto (INT, FK):** Vincula la línea con el producto vendido.
- **cantidad (INT, NOT NULL):** Cantidad de unidades compradas de ese producto.
- **precio_unitario (DECIMAL(10,2), NOT NULL):** Precio histórico al que se vendió el producto en ese momento exacto.

Relaciones e Integridad Referencial

El modelo se basa en tres relaciones estratégicas:

1. **Clientes a Ventas (1:N):** Un cliente puede realizar múltiples compras a lo largo del tiempo, pero cada ticket de venta pertenece a un único cliente. Se aplica `ON DELETE RESTRICT` para evitar borrar un cliente si este ya tiene un historial de compras.
2. **Ventas a Detalle de Ventas (1:N):** Una venta contiene múltiples renglones de artículos. Se aplica `ON DELETE CASCADE` para que, si se anula o elimina una venta principal, automáticamente se limpien todos sus renglones asociados.
3. **Productos a Detalle de Ventas (1:N):** Un producto puede ser incluido en muchos detalles de ventas diferentes. Se restringe su eliminación si ya fue vendido para no romper las estadísticas del negocio.

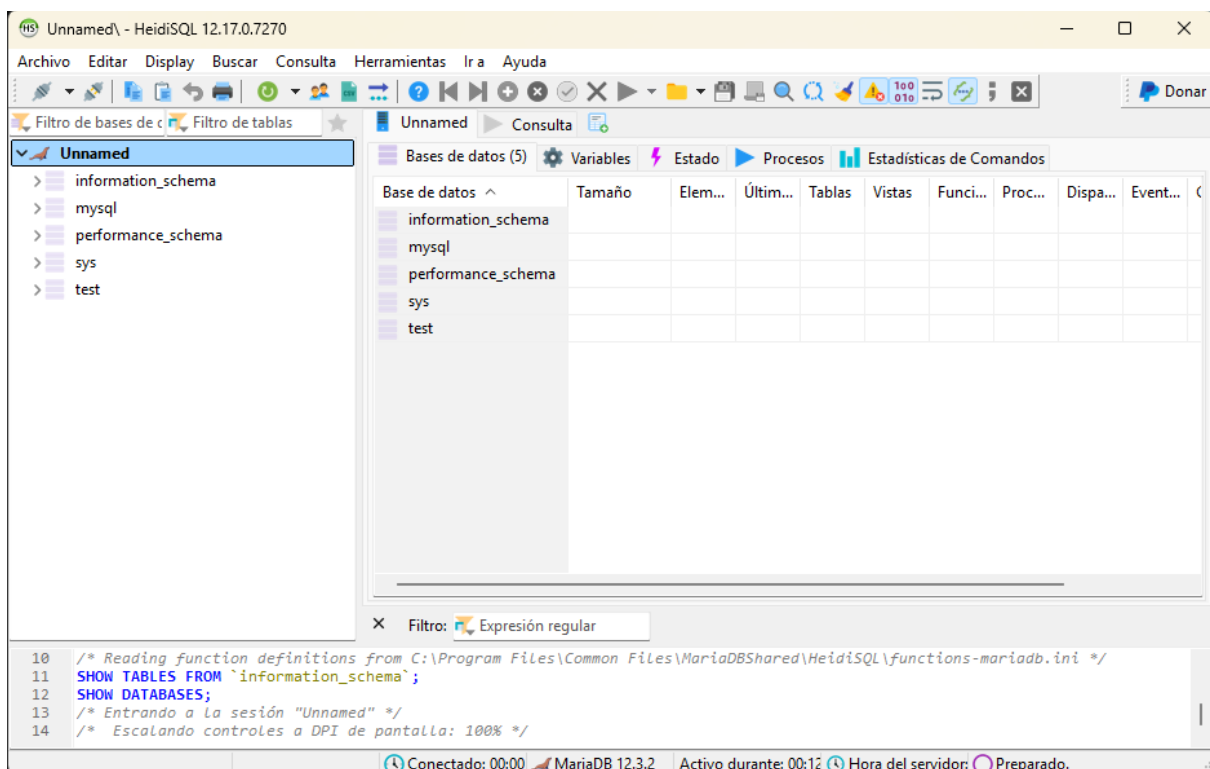
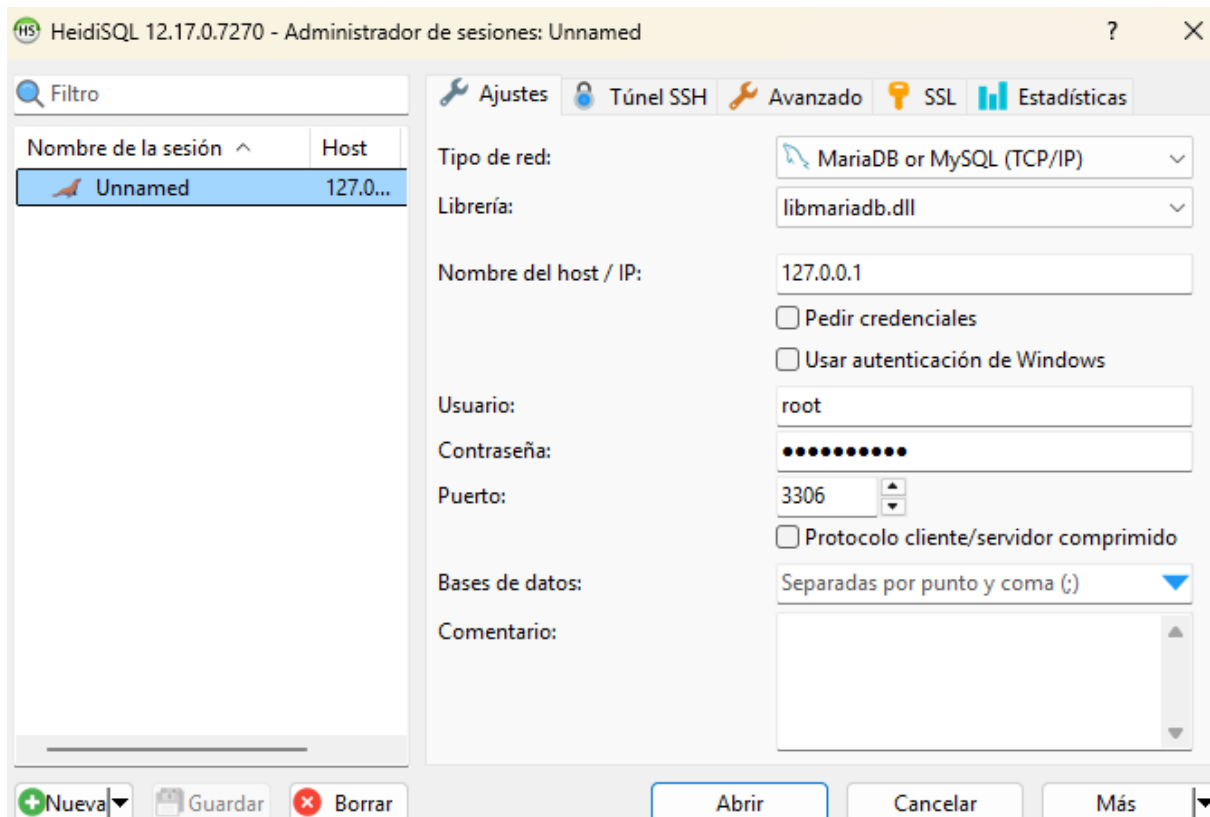
Aplicaciones Prácticas y Escenarios de Uso

Esta base de datos es ideal para:

- **Tiendas con locales físicos:** Controlar cajas diarias y emitir comprobantes.
- **Sistemas de Auditoría:** Conocer qué empleado vendió qué artículo y a qué precio exacto en una fecha determinada.
- **Estrategias de Marketing:** Analizar cuál es el producto más vendido o qué clientes no realizan compras hace meses para enviarles promociones.

3 Generar la Nueva Base de Datos en HeidiSQL

(ANTES QUE NADA LE PUSE CONTRASEÑA PORQUE ME LA PIDIO EN LA INSTALACION LA CONTRASEÑA ES “bautista08”)



The image shows two screenshots of the HeidiSQL 12.17.0.7270 interface. The top screenshot displays the first three steps of a SQL script: creating the database 'tienda_videojuegos', using it, and creating the 'clientes' and 'productos' tables. The bottom screenshot displays the next two steps: creating the 'ventas' table with a foreign key to 'clientes' and the 'detalle_ventas' table with foreign keys to both 'ventas' and 'productos'.

```
1 -- 1 Crear La base de datos si no existe
2 CREATE DATABASE IF NOT EXISTS tienda_videojuegos CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
3
4 -- Usar La base de datos que hice
5 USE tienda_videojuegos;
6
7 -- 2 Crear La tabla de Clientes
8 CREATE TABLE clientes (
9     id_cliente INT AUTO_INCREMENT,
10    dni VARCHAR(20) NOT NULL UNIQUE,
11    nombre VARCHAR(100) NOT NULL,
12    apellido VARCHAR(100) NOT NULL,
13    email VARCHAR(100),
14    PRIMARY KEY (id_cliente)
15 );
16
17 -- 3 Crear La tabla de Productos
18 CREATE TABLE productos (
19     id_producto INT AUTO_INCREMENT,
20    nombre_producto VARCHAR(100) NOT NULL,
21    precio DECIMAL(10,2) NOT NULL,
22    stock INT NOT NULL DEFAULT 0,
23    PRIMARY KEY (id_producto)
24 );
25
26
27 -- 4 Crear La tabla de Ventas
28 CREATE TABLE ventas (
29     id_venta INT AUTO_INCREMENT,
30    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
31    id_cliente INT,
32    PRIMARY KEY (id_venta),
33    CONSTRAINT FK_ventas_clientes FOREIGN KEY (id_cliente)
34        REFERENCES clientes(id_cliente)
35        ON DELETE RESTRICT
36        ON UPDATE CASCADE
37 );
38
39 -- 5 Crear La tabla Detalle de Ventas
40 CREATE TABLE detalle_ventas (
41     id_detalle INT AUTO_INCREMENT,
42    id_venta INT,
43    id_producto INT,
44    cantidad INT NOT NULL,
45    precio_unitario DECIMAL(10,2) NOT NULL,
46    PRIMARY KEY (id_detalle),
47    CONSTRAINT FK_detalle_ventas_ventas FOREIGN KEY (id_venta)
48        REFERENCES ventas(id_venta)
49        ON DELETE CASCADE
50        ON UPDATE CASCADE,
51    CONSTRAINT FK_detalle_ventas_productos FOREIGN KEY (id_producto)
52        REFERENCES productos(id_producto)
53        ON DELETE RESTRICT
54        ON UPDATE CASCADE
55 );
```

(codigo para hacer la base)

(explicacion de codigo + sitios de ayuda)

CREATE DATABASE IF NOT EXISTS tienda_videojuegos CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci; USE tienda_videojuegos;

- **CREATE DATABASE IF NOT EXISTS:** Crea el contenedor principal de los datos. La cláusula IF NOT EXISTS previene errores en caso de que vuelvas a ejecutar el script si la base de datos ya fue creada.
- **CHARACTER SET utf8mb4 y COLLATE utf8mb4_general_ci:** Define la codificación de caracteres. Usar utf8mb4 asegura soporte completo para textos en español (acentos, eñes) e incluso emojis. *_ci* significa *Case Insensitive* (no distingue mayúsculas de minúsculas en las búsquedas).

- **USE:** Le indica al servidor MariaDB que todas las consultas y tablas que se creen a continuación se ejecutarán dentro de esa base de datos de forma activa.

Referencia: <https://mariadb.com/docs/server/reference/sql-statements/data-definition/create/create-database>

```
CREATE TABLE clientes ( id_cliente INT AUTO_INCREMENT, dni VARCHAR(20) NOT NULL UNIQUE, ... PRIMARY KEY (id_cliente) );
```

- **INT AUTO_INCREMENT:** Define el identificador como un número entero que MariaDB incrementará automáticamente (+1) cada vez que se inserte un registro.
- **NOT NULL:** Restricción que obliga a que el campo tenga un valor sí o sí; no permite celdas vacías.
- **UNIQUE:** Garantiza que no se puedan registrar dos filas con el mismo valor (por ejemplo, dos clientes con el mismo DNI).
- **PRIMARY KEY:** Define la clave primaria de la tabla, asegurando la identidad única de cada fila y acelerando las búsquedas en el sistema.
- **DECIMAL(10, 2) (en la tabla productos):** Tipo de dato óptimo para almacenar dinero. Reserva hasta 10 dígitos en total, con 2 dígitos destinados estrictamente a los centavos, evitando errores de redondeo que ocurren con tipos como FLOAT.
- **Referencia:** [MariaDB - CREATE TABLE](#) y [MariaDB - Data Types](#)

```
CONSTRAINT FK_ventas_clientes FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE RESTRICT ON UPDATE CASCADE
```

- **CONSTRAINT FOREIGN KEY:** Define una restricción de clave foránea. Vincula una columna de la tabla actual con la clave primaria de otra tabla, impidiendo que se ingresen datos inexistentes (por ejemplo, registrar una venta a un id_cliente que no existe).

- **ON DELETE RESTRICT:** Regla de integridad que prohíbe borrar un registro de la tabla padre (ej. un Cliente) si existen registros asociados en la tabla hijo (ej. Ventas). Esto evita dejar "ventas huérfanas" en el sistema.
- **ON UPDATE CASCADE:** Si por algún motivo cambia el ID de un cliente en la tabla principal, ese cambio se replica automáticamente ("en cascada") en todas las ventas donde aparecía ese cliente, manteniendo los datos sincronizados.
- **ON DELETE CASCADE (en detalle_ventas):** Si se decide eliminar una factura completa de la tabla ventas, se eliminarán de forma automática todas sus líneas de artículos de la tabla detalle_ventas, limpiando la base de datos sin dejar residuos.
- **Referencia:** [MariaDB - Foreign Keys](https://mariadb.com/kb/en/foreign-keys/)
- **Manual de Referencia General de MariaDB:**
<https://mariadb.com/kb/en/documentation/>

(diferentes pruebas que se fueron haciendo)

The screenshot shows the HeidiSQL interface with the following components:

- Left Panel (Database Tree):** Shows the 'tienda_videojuegos' database selected, containing tables: clientes (32,0 KiB), detalle_ventas (48,0 KiB), productos (16,0 KiB), and ventas (32,0 KiB).
- Central Panel (SQL Editor):** Contains two SQL queries:


```

-- 4 Crear La tabla de Ventas
CREATE TABLE ventas (
  id_venta INT AUTO_INCREMENT,
  fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  id_cliente INT,
  PRIMARY KEY (id_venta),
  CONSTRAINT FK_ventas_clientes FOREIGN KEY (id_cliente)
    REFERENCES clientes(id_cliente)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

-- 5 Crear La tabla Detalle de Ventas
CREATE TABLE detalle_ventas (
  id_detalle INT AUTO_INCREMENT,
  id_venta INT,
  id_producto INT,
  cantidad INT NOT NULL,
  precio_unitario DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (id_detalle),
  CONSTRAINT FK_detalle_ventas_ventas FOREIGN KEY (id_venta)
    REFERENCES ventas(id_venta)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
      
```
- Right Panel (Filter ...):** Shows a list of filters including Columns, Funciones SQL, Palabras clave SQL, Fragmentos, Historial de consultas, Perfil de consultas, and Parámetros Enlazados.
- Bottom Panel (Query Results):** Displays the output of the SQL queries:


```

26 SHOW TABLE STATUS FROM `tienda_videojuegos`;
27 SHOW FUNCTION STATUS WHERE Db = `tienda_videojuegos`;
28 SHOW PROCEDURE STATUS WHERE Db = `tienda_videojuegos`;
29 SHOW TRIGGERS FROM `tienda_videojuegos`;
30 SELECT *, EVENT_SCHEMA AS `Db`, EVENT_NAME AS `Name` FROM INFORMATION_SCHEMA.`EVENTS` WHERE EVENT_SCHEMA=`tienda_videojuegos`
      
```

(se creó)

```
USE tienda_videojuegos;

-- Insertar Clientes de prueba
INSERT INTO clientes (dni, nombre, apellido, email) VALUES
('45123456', 'Juan', 'Pérez', 'juan.perez@email.com'),
('46987654', 'María', 'Gómez', 'maria.gomez@email.com'),
('44555666', 'Lucas', 'Rodríguez', 'lucas.rodri@email.com');

-- Insertar Productos
INSERT INTO productos (nombre_producto, precio, stock) VALUES
('Joystick Inalámbrico PS5', 85000.00, 15),
('Teclado Mecánico RGB', 45000.00, 20),
('Elden Ring - PC Steam', 35000.00, 50),
('Monitor Gamer 24" 144Hz', 220000.00, 8);
```

(como ya se creo todo yo al añadir mas codigo como el que se ve te salta un error de que clientes ya esta creado asique se tiene que hacer en una nueva consulta o sobre escribir el codigo que ya esta guardado)

clientes	32,0 KiB	Columnas: + Agregar - Borrar ▲ Subir ▼ Bajar
detalle_ventas	48,0 KiB	
productos	16,0 KiB	
ventas	32,0 KiB	

detalle_ventas	48,0 KiB	Columnas: + Agregar - Borrar ▲ Subir ▼ Bajar
productos	16,0 KiB	
ventas	32,0 KiB	

clientes	32,0 KiB	Columnas: + Agregar - Borrar ▲ Subir ▼ Bajar
detalle_ventas	48,0 KiB	
productos	16,0 KiB	
ventas	32,0 KiB	

tienda_videojuegos	128,0 KiB	Columnas: + Agregar - Borrar ▲ Subir ▼ Bajar
clientes	32,0 KiB	
detalle_ventas	48,0 KiB	
productos	16,0 KiB	
ventas	32,0 KiB	

(verificacion de tablas creadas)


```

USE tienda_videojuegos;

-- 1. Registramos que el cliente con id_cliente = 1 (Juan Pérez) hace una compra hoy
INSERT INTO ventas (id_cliente) VALUES (1);

-- 2. Cargamos lo que compró Juan en esa venta (id_venta = 1):
-- Compró 1 Joystick (id_producto = 1) y 2 juegos de Elden Ring (id_producto = 3)
INSERT INTO detalle_ventas (id_venta, id_producto, cantidad, precio_unitario) VALUES
(1, 1, 1, 85000.00),
(1, 3, 2, 35000.00);

-- 3. PRUEBA EL REPORTE MULTITABLA |
SELECT
    v.id_venta AS 'Nro Factura',
    v.fecha AS 'Fecha Compra',
    CONCAT(c.apellido, ' ', c.nombre) AS 'Cliente',
    p.nombre_producto AS 'Artículo',
    dv.cantidad AS 'Cant',
    dv.precio_unitario AS 'Precio Unitario',
    (dv.cantidad * dv.precio_unitario) AS 'Subtotal'
FROM detalle_ventas dv
JOIN ventas v ON dv.id_venta = v.id_venta
JOIN clientes c ON v.id_cliente = c.id_cliente
JOIN productos p ON dv.id_producto = p.id_producto;

```

(paso 4 de los join)

The screenshot shows a database management tool interface. On the left, there is a sidebar with a tree view of the database structure. The 'tienda_videojuegos' database is selected, and its tables are listed: 'clientes' (32.0 KIB), 'detalle_ventas' (48.0 KIB), 'productos' (16.0 KIB), and 'ventas' (32.0 KIB). The 'detalle_ventas' table is currently selected.

The main window displays the data for the 'detalle_ventas' table. The table has 5 columns: 'id_detalle', 'id_venta', 'id_producto', 'cantidad', and 'precio_unitario'. The data is as follows:

#	1 id_detalle	2 id_venta	3 id_producto	4 cantidad	5 precio_unitario
1	1	1	1	1	85.000,0
2	2	1	3	2	35.000,0

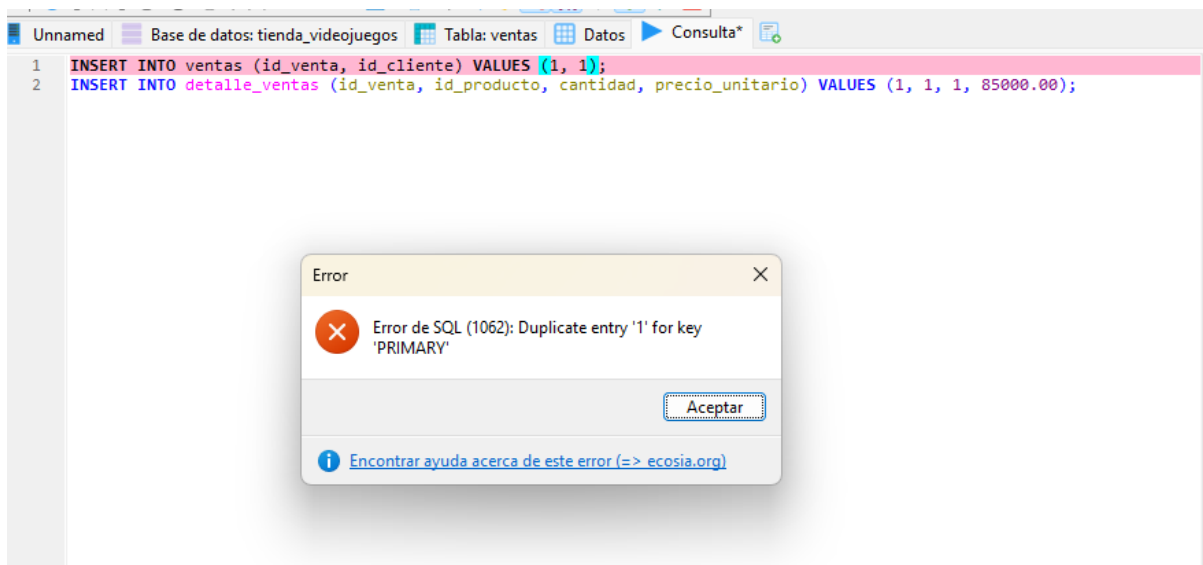
tienda_videojuegos.productos: 4 filas en total (exacto)

#	1 id_producto	2 nombre_producto	3 precio	4 stock
1	1	Joystick Inalámbrico PS5	85.000,0	15
2	2	Teclado Mecánico RGB	45.000,0	20
3	3	Elden Ring - PC Steam	35.000,0	50
4	4	Monitor Gamer 24" 144Hz	220.000,0	8

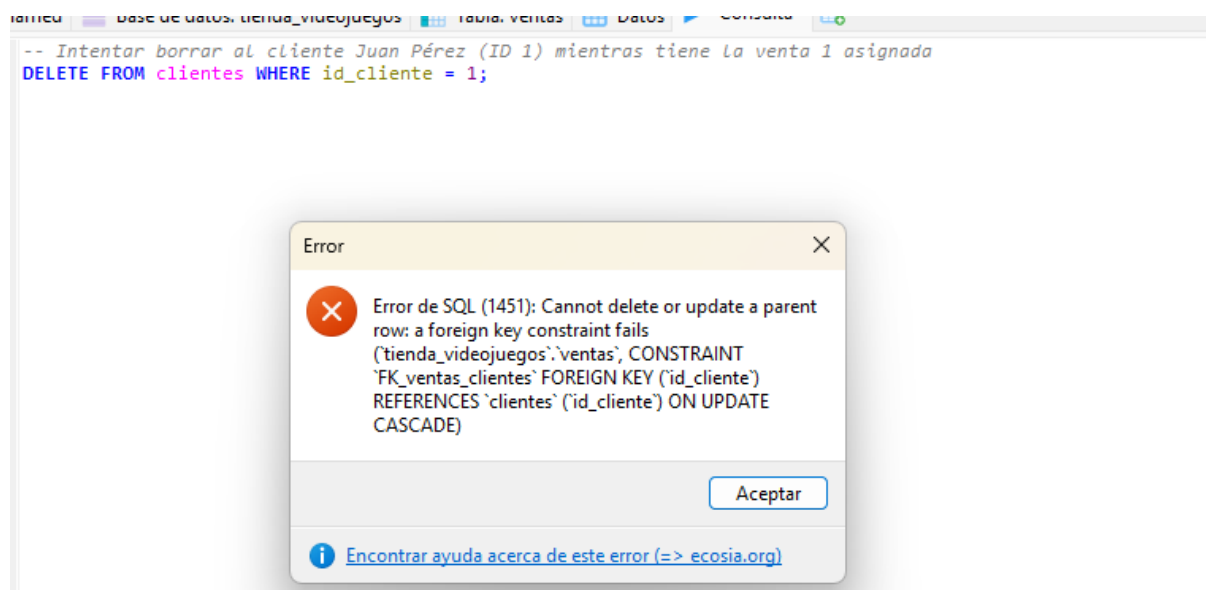
tienda_videojuegos.ventas: 1 fila en total (exacto)

#	1 id_venta	2 fecha	3 id_cliente
1	1	2026-06-30 05:58:47	1

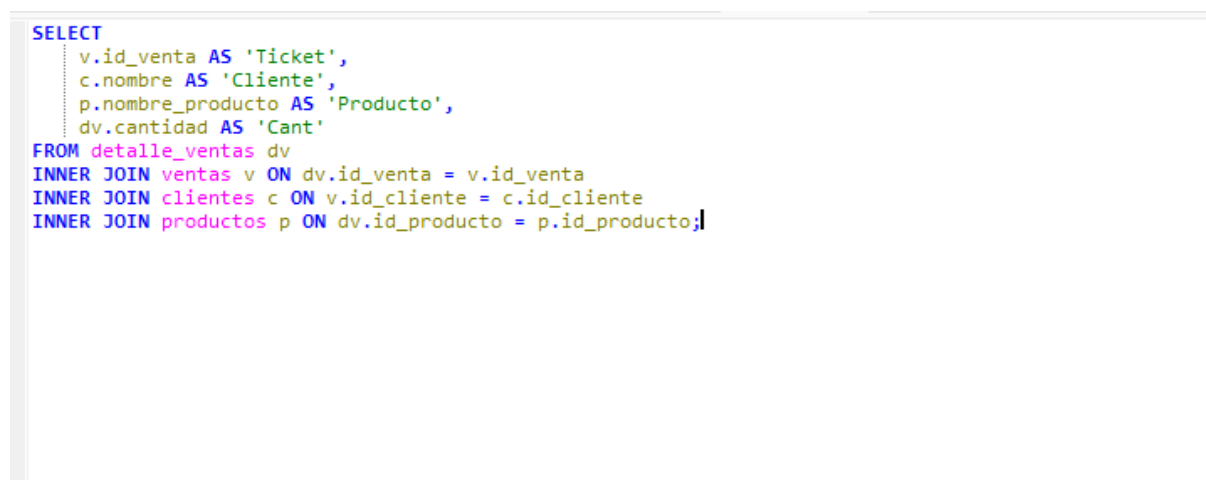
(se van a ir añadiendo datos conforme se vayan probando)



muestra qué pasa cuando querés hacer algo ilegal en el sistema. se intento borrar un cliente o producto que tiene datos asociados.



MariaDB frena la operación gracias a la regla ON DELETE RESTRICT. Esto evita que borres un cliente sin querer y dejes facturas sin dueños en el sistema.



El INNER JOIN es una consulta que busca la **coincidencia exacta** entre las tablas. Si un dato no tiene su pareja en la otra tabla, directamente **no aparece en el resultado**.

- Cruzamos detalle_ventas, ventas, clientes y productos. Como el único cliente que efectivamente compró algo fue Juan Pérez, la consulta te devolvió únicamente sus **2 renglones** (el joystick y el juego Elden Ring).
- **¿Por qué no aparecieron María ni Lucas?** Porque ellos todavía no tienen ninguna factura cargada en la tabla ventas. Como no hay coincidencia, el INNER JOIN los ignora por completo.
- **Enlace oficial para la documentación:** [MariaDB - INNER JOIN](#)

The screenshot shows a database IDE with a SQL query editor and an error dialog box. The query editor contains the following SQL code:

```
1 USE tienda_videojuegos;
2
3 -- 1. PRUEBA DEL LEFT JOIN (Trae los clientes que NO compraron nada)
4 SELECT
5     c.id_cliente,
6     CONCAT(c.apellido, ', ', c.nombre) AS 'Cliente',
7     v.id_venta AS 'Nro Compra'
8 FROM clientes c
9 LEFT JOIN ventas v ON c.id_cliente = v.id_cliente
10 WHERE v.id_venta IS NULL;
11
12 -- 2. PRUEBA DE INTEGRIDAD (Intentar borrar un cliente con ventas activas)
13 -- Esto tiene que tirar un error en rojo a propósito para demostrar que la base de datos es segura.
14 DELETE FROM clientes WHERE id_cliente = 1;
```

The error dialog box displays the following message:

Error de SQL (1451): Cannot delete or update a parent row: a foreign key constraint fails ('tienda_videojuegos`.`ventas`, CONSTRAINT `FK_ventas_clientes` FOREIGN KEY (`id_cliente`) REFERENCES `clientes` (`id_cliente`) ON UPDATE CASCADE)

Below the error dialog, the results pane shows the output of the first query:

id_cliente	Cliente
1	Pérez
2	Gómez

El LEFT JOIN es distinto: le da prioridad absoluta a la tabla que está a la izquierda (en mi caso, la tabla clientes). Trae a **todos** los clientes de esa tabla, sin importar si compraron algo o no. Si no tienen compras asociadas, los campos de la tabla derecha (ventas) te van a aparecer vacíos (NULL).

- **¿Qué pasó en mi código?** Al poner la condición `WHERE v.id_venta IS NULL`, lo que hice fue decirle a MariaDB: *"Traeme todos los clientes, pero mostrame solo los que tengan la parte de ventas vacía"*.
- **¿Cuál fue el resultado?** Te aparecieron exactamente **2 filas**: María Gómez y Lucas Rodríguez. Esto te demostró de forma matemática quiénes son tus clientes inactivos.